Mohsin Iban Hossain

AIUB, Computer Graphics Notes

COMPUTER GRAPHICS

Table of Contents

Sl. No	Topic	Pages
1	Introduction to Computer Graphics	03-11
2	Practice Questions	11-16
3	Scan Conversation	17-19
4	DDA Practice Questions	20-22
5	Bresenham's Mid-Point Line Algorithm	23-28
6	Bresenham's Midpoint Circle Algorithm	29-34
7	MCQ Practice	35-45

INTRODUCTION TO COMPUTER GRAPHICS

Pixel

- A **pixel** is the smallest part of a picture, holding color and brightness.
- Pixels are arranged in a **grid**, and more pixels mean a **clearer image** (higher resolution).
- Image size = **width × height in pixels**, affected by **Pixels Per Inch (PPI)**.
- Pictures appear when pixels combine with different colors.

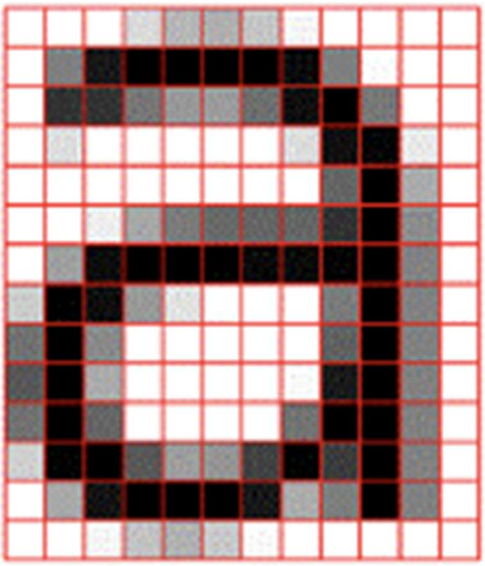


Fig-1 (a): Image

Fig-1 (b): Pixel Representation

Graphics Images

➔ There are two types of graphics image:

1. **Raster Image**
2. **Vector Image**

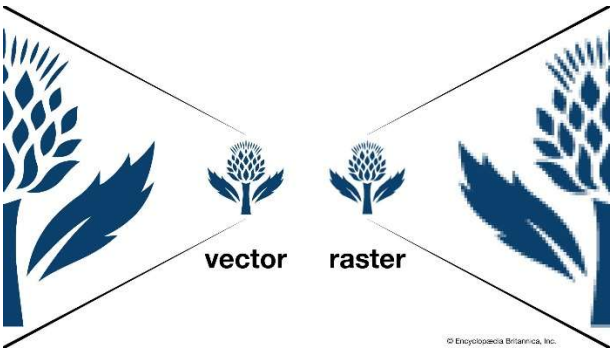


Fig-2: Graphics Images

Raster Image

- **Bitmap structure**: Images are made of pixels in a grid.
- **Fixed color**: Each pixel holds one color.
- **Resolution-dependent**: Scaling up results in pixilation.
- **Best for**: Photos and detailed images.
- **Common formats**: JPEG, PNG, GIF, BMP, TIFF.
- **Examples**: Digital photos, scanned images, screenshots.

Vector Image

- **Vector graphics**: Made of mathematical equations, not pixels.
- **Resolution-independent**: Can be resized without losing quality.
- **Best for**: Logos, illustrations, and text-based graphics.
- **Common formats**: SVG, AI, EPS, PDF.
- **Examples**: Company logos, icons, infographics.

Formation of Raster Image & Vector Image

Raster Image	Vector Image
• **Grid Structure**: Raster images are made of a grid of pixels, like a **mosaic.** • **Color (RGB Model)**: Each pixel gets a color from Red, Green, and Blue (0-255 each). • **Resolution & Detail**: ⇨ More pixels = sharper image. ⇨ fewer pixels = pixilation when enlarged. • **Display on Screen**: Pixels light up in colors, and the eye blends them into a smooth image.	• **Vector images use math**: Made of points, lines, and curves. • **Scalable & sharp**: No pixilation when resized. • **Color & fill**: Supports solid colors, gradients, and patterns. • **Display on Screen**: ⇨ Rasterization (Vector to raster) ⇨ Mapped with display pixel coordinate.

Color Model

➔ A **color model** is a mathematical representation of colors used in digital images, defining how colors are created and stored in a computer.

- Common models: **RGB** (Red, Green, Blue) & **CMYK** (Cyan, Magenta, Yellow, Black).
- Colors are represented through **color space**, defining the range of possible colors.

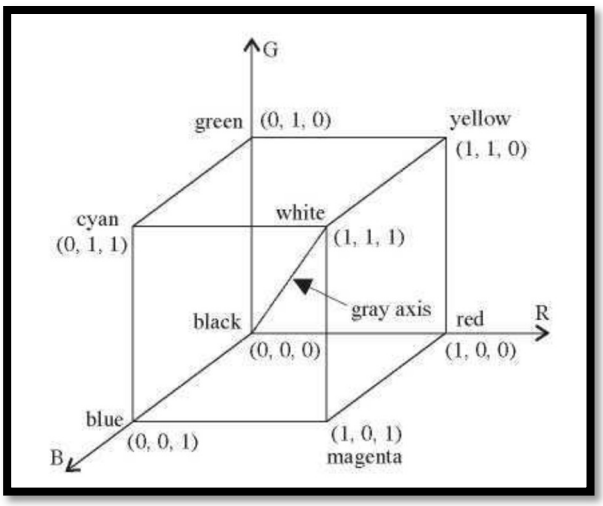


Fig-3 (a): RGB Color Space

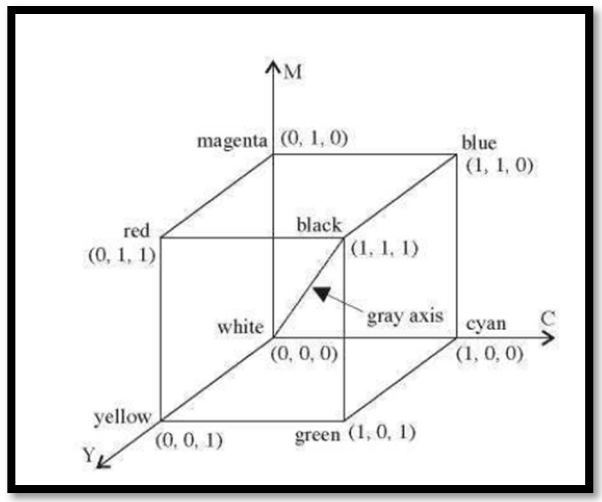


Fig-3 (b): CMY Color Space

RGB Color Model

Definition:

The **RGB color model** is an additive color model where colors are created by mixing three primary colors—Red (R), Green (G), and Blue (B)—at varying intensity levels from 0 (lowest) to 1 (highest).

Key Points:

- The RGB model is based on an **additive process**, starting with **black** and adding light.
- Different intensity combinations of **R, G, and B** produce various colors.
- The RGB color space forms a **cube**, where each color is represented by coordinates (r, g, b).
- Example color coordinates:
    - **Black** → (0,0,0)
    - **White** → (1,1,1)
    - **Yellow** → (1,1,0)
    - **Gray (midway)** → (0.7,0.7,0.7)

CMY Color Model

Definition:

The **CMY color model** is a subtractive color model that defines colors using three primary colors—Cyan (C), Magenta (M), and Yellow (Y). It is **complementary** to the RGB model.

Key Points:

- The CMY model uses a **subtractive process**, starting with **white** and removing colors.
- Removing **red** leaves **cyan**, removing **green** leaves **magenta**, and removing **blue** leaves **yellow**.
- The CMY color space is represented using **(C, M, Y) coordinates**.
- **Example:** Subtracting red from white results in **cyan** (combination of green and blue).

Conversion Formula between the two-color models

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} - \begin{pmatrix} C \\ M \\ Y \end{pmatrix} \qquad \begin{pmatrix} C \\ M \\ Y \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} - \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

RGB to CMY Conversion

$$C = 1 - \frac{Color.R}{255.0}$$

$$M = 1 - \frac{Color.G}{255.0}$$

$$Y = 1 - \frac{Color.B}{255.0}$$

CMY to RGB Conversion

$$R = (1 - C) * 255.0$$

$$G = (1 - M) * 255.0$$

$$B = (1 - Y) * 255.0$$

Color Representation Techniques

Definition:

In **digital image processing**, different methods are used to store and access pixel color information. Two common methods are **Direct Coding** and **Lookup Tables (LUTs)**.

Direct Coding:

- Each pixel is allocated a fixed amount of storage space for its color.
- Example: **3-bit representation** (1 bit per primary color: R, G, B).
- Each primary can be **0 (off)** or **1 (on)**, allowing **8 possible colors** (corners of the RGB color cube).

Direct Coding Example: Color Palette

- **3-bit RGB:**
  - Uses **1 bit** per primary color (**R, G, B**).
  - Each component is either **on (1)** or **off (0)**.
  - Results in **8 colors**: $(2^1)^3 = 8$
- **6-bit RGB:**
  - Uses **2 bits** per primary color.
  - Each component has **4 intensity levels** ($2^2 = 4$).
  - Results in **64 colors**: $(2^2)^3 = 64$
- **General Formula:**

Total Colors = $(2^n)^3$ [where **n** is the number of bits per primary color.]

LookUp Table (LUT)

➔ A **Lookup Table (LUT)** is an indirect method of storing color information where pixel values do not directly encode colors but act as **indices** pointing to a predefined table of colors.

How LUT Works:

1. Each pixel stores an **index value** instead of actual color information.
2. The index references a **color entry** in the lookup table.
3. The lookup table contains **predefined color values**, usually in RGB format.
4. When displaying the image, the system maps the pixel indices to the actual color values stored in the table.

Advantages of LUT:

- **Efficient memory usage:** Instead of storing full RGB values for every pixel, LUTs allow storing only index values, reducing storage requirements.
- **Flexible color mapping:** Easily modify colors by updating the table without changing the pixel data.
- **Useful for limited color displays:** Common in systems with **8-bit color depth**, where 256 colors are stored in a LUT.

Example of LUT Usage:

- **8-bit LUT system:**
  - Each pixel holds an **8-bit index** (0-255).
  - The lookup table stores **256 RGB color values**.
  - The system uses the pixel index to retrieve the corresponding RGB color.

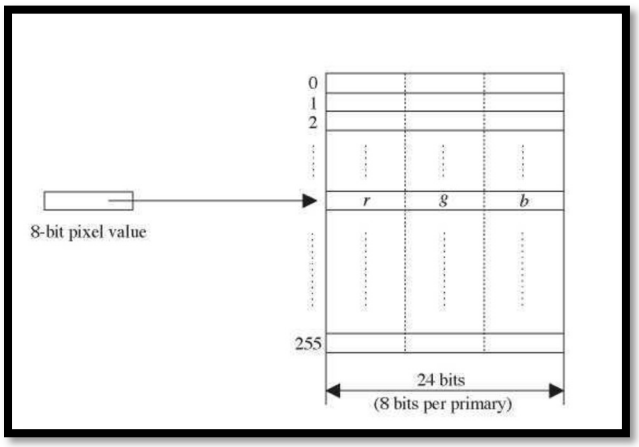


Fig-4: Lookup table

Printing

- A printer deposits **color pigments** onto **paper** to create colors.
- Uses **cyan, magenta, and yellow** to control **red, green, and blue** light.
- **Pigments remain as tiny dots** rather than blending in some methods.
- **Halftone technique** creates **shades and gradients** using **varying dot sizes and spacing**.
- Commonly used in **printing, newspapers, and comics**.

Halftone Technique

- **Halftone** is the technique that simulates continuous tone imagery using dots.
- **Dots can be varied either**
    - **in size**
    - **in shape or**
    - **in spacing**
- **Halftone generates a gradient like effect.**

Halftone Image

- **Halftone image** consists of **discrete dots**, not continuous tones.
- **Dots blur together** from a distance, creating an illusion of continuous shapes.
- **Dots are arranged** at a **45° screen angle** to the horizon.
- **Dot size is inversely proportional** to the **intensity level**.

Practice Questions

Problem1: Explain how the equations $y = x + 3$ can be used to create a vector image?

➔ The equation $y = x + 3$ represents a straight line. To create a vector image, plot the line on a coordinate plane, and each point on the line corresponds to a $vector\ v = \langle x, x + 3 \rangle$, representing the position on the line.

Or, the equation $y = x + 3$ represents a straight line with a slope of 1 and a y-intercept of 3. To create a vector image:

1. **Graphical Representation**: Plot the equation on a coordinate plane, where the x-axis and y-axis represent the horizontal and vertical components, respectively. Each point on the line corresponds to a vector from the origin (0, 0) to the point (x, y), forming a vector field along the line.
2. **Vector Equation**: A vector image can be created by treating the points on the line as vectors. For any point (x, y), the vector can be represented as $v = \langle x, x + 3 \rangle$, which is a sequence of vectors along the line $y = x + 3$

Problem2: Show mathematically how you can create a different color Palettes

➔ To mathematically create different color palettes, we use the formula:

Total Color = $(2^n)^3$ [where **n** is the number of bits per primary color.]

Varying Bit Depths for Different Palettes

- **3-bit RGB ($n = 1$)**:

Total Color Palettes = $(2^1)^3$ = **8** colors

- **6-bit RGB($n = 2$)**:

Total Color Palettes = $(2^2)^3 = \mathbf{64}$ colors

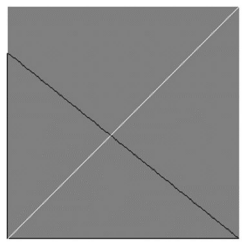
Problem3: write down the purpose of the following functions

Function/Parameter	Purpose
`glutDisplayFunc(display);`	Registers the **`display`** function as the display callback, which is called whenever the window needs to be redrawn.
`scene in (a);`	Represents the scene or graphical content that is being rendered in part (a) of the program.
`gluOrtho2D(l, r, b, t);`	Defines a 2D orthographic projection with left (`l`), right (`r`), bottom (`b`), and top (`t`) clipping boundaries.
`GL_LINE`	Used in OpenGL to specify that a shape should be drawn as a line or a series of connected line segments.
`l` and `b` in (c)	Parameters defining the left (`l`) and bottom (`b`) boundaries of the viewport or coordinate system in part (c) of the program.

OR

Function/Parameter	Purpose
`glutDisplayFunc(display);`	Sets the display callback function.
`scene in (a);`	likely refers to rendering objects.
`gluOrtho2D(l, r, b, t);`	Sets a 2D orthographic projection.
`GL_LINE`	Used to draw lines in OpenGL.
`l` and `b` in (c)	`l` (left) and `b` (bottom) set the viewport limits.

Problem4: Suppose you need to design the given image using glut. Write the code to represent the image only showing the vertex plotting part using the given sample. You must show all the elements given in the samples.



Sample:

```
glBegin(PRIMITIVE);
    glVertex2f(x1, y1);
    glVertex2f(x2, y2);
    glVertex2f(x3, y3);
    ...
    ...
glEnd();
```

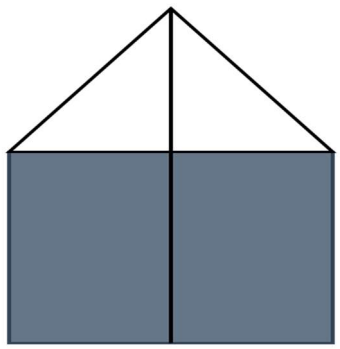
Ans:

```
// Draw the square using GL_QUADS
glColor3f(0.5f, 0.5f, 0.5f); // Gray color
glBegin(GL_QUADS); // PRIMITIVE is GL_QUADS
    glVertex2f(-0.5f, -0.5f);
    glVertex2f(0.5f, -0.5f);
    glVertex2f(0.5f, 0.5f);
    glVertex2f(-0.5f, 0.5f);
glEnd();

// Draw a triangle using GL_ LINE_LOOP
glColor3f(0.0f, 0.0f, 0.0f); // Black color
glBegin(GL_LINE_LOOP);
    glVertex2f(-0.5f, -0.5f);
    glVertex2f(0.5f, -0.5f);
    glVertex2f(-0.5f, 0.3f);
glEnd();

// Draw a diagonal line using GL_LINES
glColor3f(1.0f, 1.0f, 1.0f); // White color
glLineWidth(0.5f);
glBegin(GL_LINES);
    glVertex2f(-0.5f, -0.5f);
    glVertex2f(0.5f, 0.5f);
glEnd();
```

Problem5: Suppose you need to design the given image using glut. Write the code to represent the image only showing the vertex plotting part using the given sample. You must show all the elements given in the samples.



Sample:

```
glBegin(PRIMITIVE);
    glVertex2f(x1, y1);
    glVertex2f(x2, y2);
    glVertex2f(x3, y3);
    ...
    ...
glEnd();
```

Ans:

```
// Draw the square using GL_QUADS
glColor3f(0.5f, 0.5f, 0.5f); // Gray color
glBegin(GL_QUADS); // PRIMITIVE is GL_QUADS
glVertex2f(-0.5f, -0.5f);
glVertex2f(0.5f, -0.5f);
glVertex2f(0.5f, 0.5f);
glVertex2f(-0.5f, 0.5f);
glEnd();

// Draw a triangle using GL_LINE_LOOP
glColor3f(0.0f, 0.0f, 0.0f); // Black color
glBegin(GL_LINE_LOOP);
glVertex2f(0.5f, 0.5f);
glVertex2f(0.0f, 1.0f);
glVertex2f(-0.5f, 0.5f);
glEnd();

// Draw a diagonal line using GL_LINES
glColor3f(0.0f, 0.0f, 0.0f); // White color
glLineWidth(0.5f);
glBegin(GL_LINES);
glVertex2f(0.0f, -0.5f);
glVertex2f(0.0f, 1.0f);
glEnd();
```

Common OpenGL Primitives

⇨ When calling `glBegin(PRIMITIVE);` WE need to replace `PRIMITIVE` with one of the following:

Primitive	Description
`GL_POINTS`	Draws individual points
`GL_LINES`	Draws separate line segments (every two vertices form a line)
`GL_LINE_STRIP`	Draws a connected sequence of lines
`GL_LINE_LOOP`	Same as `GL_LINE_STRIP` but connects the last vertex to the first
`GL_TRIANGLES`	Draws separate triangles (every three vertices form a triangle)
`GL_TRIANGLE_STRIP`	Connected triangles that share vertices
`GL_TRIANGLE_FAN`	Like `GL_TRIANGLE_STRIP`, but with a shared center vertex
`GL_QUADS`	Draws separate quadrilaterals (four vertices per quad)
`GL_QUAD_STRIP`	A connected strip of quadrilaterals
`GL_POLYGON`	Draws a filled convex polygon

SCAN CONVERSATION

Scan Conversation

⇨ **Scan Conversion** is the process of converting geometric shapes (such as lines, triangles, and polygons) into a set of pixels that can be displayed on a screen. It is a key step in rasterization, ensuring that vector-based graphics are properly mapped to a raster display.

Scan Conversion in Rasterization

- **Final step of rasterization** – Converts geometric shapes into pixels.
- **Occurs after clipping** in the rendering pipeline.
- **Used in all graphics packages** at the end of rendering.
- **Maps triangles to pixels** on the screen.
- **Considers lighting and shading**, but focus is on, line scan conversion algorithms.

Rasterization

⇨ **Rasterization** is the process of converting vector graphics (shapes) into a raster image (pixels) for display, printing, or storage.

Incremental Algorithm

⇨ **Incremental Algorithm** is a line drawing method that starts from the initial pixel and progresses step by step to the final pixel.

❖ Special Cases in Line Drawing:

- **Horizontal Line:** Increment **x** by 1 to get the next pixel.
- **Vertical Line:** Increment **y** by 1 to get the next pixel.
- **Diagonal Line:** Increment both **x** and **y** by 1 to get the next pixel.
- **General Case:** Increment **x** by 1 and choose the pixel closest to the line.

Digital Differential Analyzer (DDA)

⇨ **DDA** is a graphics algorithm for interpolating points to rasterize lines, triangles, and polygons.

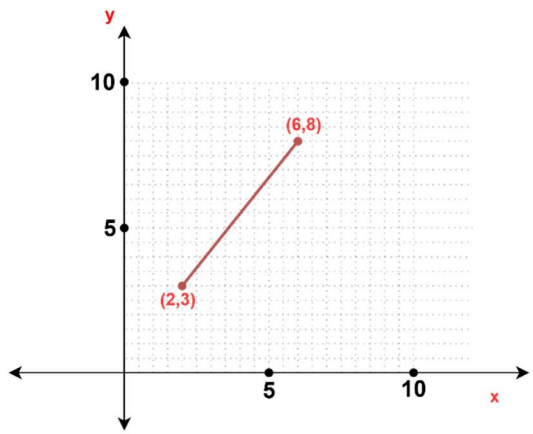
Formula

if M< 1	if M >1
➢ $x_{new} = x_{old} + 1$ ➢ $y_{new} = round(y_{old} + m)$	➢ $x_{new} = round(x_{old} + \frac{1}{m})$ ➢ $y_{new} = y_{old} + 1$

if M=1
➢ $x_{new} = x_{old} + 1$ ➢ $y_{new} = y_{old} + 1$

Practice Problem

Problem1: Use the DDA algorithm to determine the pixel positions for the given line segment from (x_1, y_1) to (x_2, y_2) as shown in the graph.



Is given,

$X_1 = 2$ $\quad$ $Y_1 = 3$

$X_2 = 6$ $\quad$ $Y_2 = 8$

We Know,

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{8-3}{6-2} = 1.25 > 1$$

if $m > 1$ then,

- $x_{new} = round(x_{old} + \frac{1}{m})$
- $y_{new} = y_{old} + 1$

x_{old}	y_{old}	x_{new}	y_{new}	x_{plot}	y_{plot}
2	**3**	$(x_{old} + \frac{1}{m}) = (2 + \frac{1}{1.25}) = 2.8$	$y_{old} + 1 = 3 + 1 = 4$	$round(2.8) = 3$	4
2.8	4	$(x_{old} + \frac{1}{m}) = (2.8 + \frac{1}{1.25}) = 3.6$	$y_{old} + 1 = 4 + 1 = 5$	$round(3.6) = 4$	5
3.6	5	4.4	6	4	6
4.4	6	5.2	7	5	7
5.2	7	6	8	**6**	**8**

Problem2: Use the **Digital Differential Analyzer (DDA) algorithm** to find and plot the intermediate points for the line segment between **(2,3) & (8,6)**.

➔ **Is given,**

$X_1 = 2$ $\quad$ $Y_1 = 3$

$X_2 = 8$ $\quad$ $Y_2 = 6$

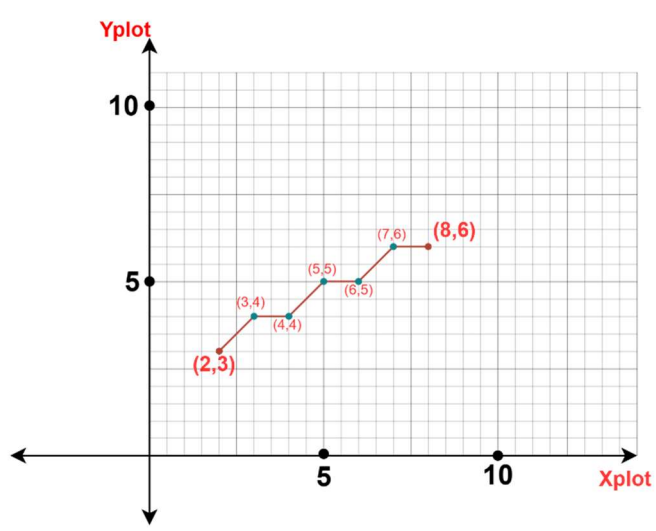
We Know,

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{6-3}{8-2} = 0.5 < 1$$

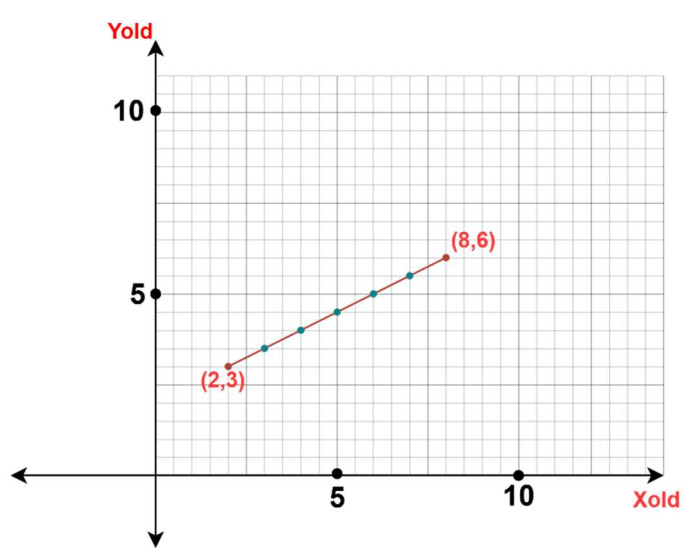
if $m < 1$ then,

- $x_{new} = x_{old} + 1$
- $y_{new} = round(y_{old} + m)$

x_{old}	y_{old}	x_{new}	y_{new}	x_{plot}	y_{plot}
2	**3**	$(x_{old} + 1) = (2+1) = 3$	$(y_{old} + m) = 3 + 0.5 = 3.5$	3	$round(3.5) = 4$
3	3.5	$(x_{old} + 1) = (3+1) = 4$	$(y_{old} + m) = 3.5 + 0.5 = 4$	4	$round(4) = 4$
4	4	5	4.5	5	5
5	4.5	6	5	6	5
6	5	7	5.5	7	6
7	5.5	8	6	**8**	**6**



We See the **Staircase (Jagged)** Effect



The actual straight line is smooth, but due to pixel approximation, it appears jagged on a digital display

Problem3: Use the **Digital Differential Analyzer (DDA) algorithm** to find and plot the intermediate points for the line segment between **A= (2,2) & B= (2,7)**.

➔ **Is given,**

$X_1 = 2$ $\quad$ $Y_1 = 2$

$X_2 = 2$ $\quad$ $Y_2 = 7$

We Know,

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{7-2}{2-2} = \frac{7-2}{0} = \boldsymbol{undefined}$$

So Theoretically,

DDA does not allow vertical lines.
it works with special adjustments to handle them.

Special Adjustments,

A= (2,2), B= (2,7)

$$\gg \Delta x = x_2 - x_1 = 2 - 2 = 0$$

$$\gg \Delta y = y_2 - y_1 = 7 - 2 = 5$$

$$\gg No.of\ step = 5$$ → [largest value between Δx & Δy]

$$\gg Step\ size\ for\ x = \frac{\Delta x}{No.of\ step} = \frac{0}{5} = 0$$

$$\gg Step\ size\ for\ y = \frac{\Delta y}{No.of\ step} = \frac{5}{5} = 1$$

x_{old}	y_{old}	x_{new}	y_{new}	x_{plot}	y_{plot}
2	**2**	2+0 = 2	2+1 = 3	2	3
2	3	2+0 = 2	3+1 = 4	2	4
2	4	2	5	2	5
2	5	2	6	2	6
2	6	2	7	**2**	**7**

Table: Intermediate Points Calculated Using DDA

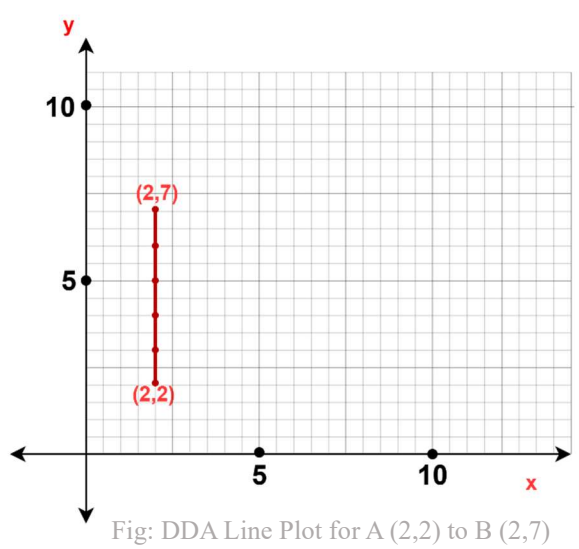


Fig: DDA Line Plot for A (2,2) to B (2,7)

Bresenham's Mid-Point Line Algorithm

- **Type:** Incremental line drawing algorithm.
- **Purpose:** Approximates a straight line between two points.
- **Approach:** Uses decision parameter to choose the next pixel.
- **Efficiency:** Avoids floating-point calculations.
- **Initial Decision Parameter:** Based on midpoint between two candidate pixels.
- **Update:** Decision parameter updated incrementally.
- **Best For:** Lines with gentle slope ($0 \le m \le 1$); can be adapted for other slopes.

Derivation of Bresenham's Midpoint Line Drawing Algorithm

⇨ **We Know,**

➢ **Line Equation:**

⇨ The general line equation between two points is:

$$\gg y = mx + c \qquad \text{or} \qquad \gg ax + by + C = 0$$

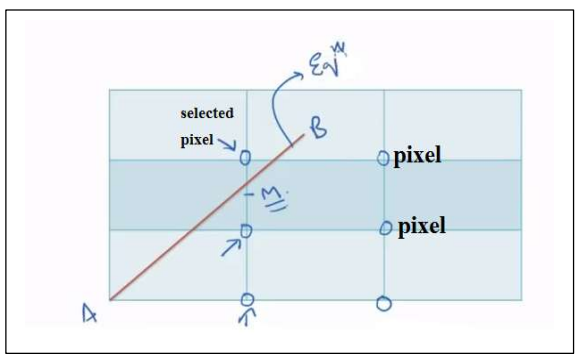
⇨ But for computer graphics, we use the **implicit form**: $f(x, y) = dy \cdot x - dx \cdot y + c$

$$\gg y = \frac{dy}{dx}x + c$$

$$\gg dy.x - dx.y + dx.c = 0$$

Where:

$a = dy;$
$b = -dx;$
$C = dx.c;$



⇨ **This function has the property:**

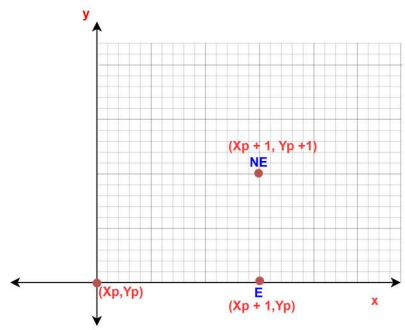
- $f(x,y) = 0$ on the line,
- $f(x,y) < 0$ below the line,
- $f(x,y) > 0$ above the line.

Decision Making Using Midpoint

≫ **Input**: Start and end points: $(x_p, y_p), (x_1, y_1)$

≫We step along x, from x_p to x_1 deciding between two pixels:

- **E (East)**: $(x_p + 1,\ y_p)$
- **NE (North_East)**: $(x_p + 1,\ y_p + 1)$



$$\therefore mid = \left(x_p + 1,\ y_p + \frac{1}{2}\right)$$

∴ f(m) = Decision Making parameter

Or $d = f(m) = f(x_p + 1,\ y_p + \frac{1}{2})$

Calculation for East (d<=0)	Calculation for North-East (d>0)
we know, $mid = \left(x_p + 1,\ y_p + \frac{1}{2}\right)$ $\gg mid_E = \left(x_p + 1 + 1,\ y_p + \frac{1}{2}\right)$ $\gg \Delta d = f\left(x_p + 2,\ y_p + \frac{1}{2}\right) - f\left(x_p + 1,\ y_p + \frac{1}{2}\right)$ $= \mathbf{a}$ $= \boldsymbol{dy}$ $\gg 2 \times \Delta d = 2dy$ $\therefore dE = 2dy$ $\boxed{\therefore d_{new} = d_{old} + dE}$ $\boxed{\therefore d_{new} = d_{old} + 2dy}$	***we know,*** $mid = \left(x_p + 1,\ y_p + \frac{1}{2}\right)$ $\gg mid_{NE} = \left(x_p + 1 + 1,\ y_p + \frac{1}{2} + 1\right)$ $\gg \Delta d = f\left(x_p + 2,\ y_p + \frac{3}{2}\right) - f\left(x_p + 1,\ y_p + \frac{1}{2}\right)$ $= \mathbf{a} + \mathbf{b}$ $= \boldsymbol{dy} - \boldsymbol{dx}$ $\gg 2 \times \Delta d = 2(dy - dx)$ $\therefore dNE = 2(dy - dx)$ $\boxed{\therefore d_{new} = d_{old} + dNE}$ $\boxed{\therefore d_{new} = d_{old} + 2(dy - dx)}$

Initial Decision Parameters
$if,$ $mid_0 = \left(x_0 + 1,\ y_0 + \frac{1}{2}\right)$ $\gg d_i = f(mid_0)$ $\gg d_i = f\left(x_0 + 1,\ y_0 + \frac{1}{2}\right)$ $= a(x_0 + 1) + b\left(y_0 + \frac{1}{2}\right) + c$ $= ax_0 + b\,y_0 + c + a + \frac{b}{2}$ $\quad \because ax_0 + b\,y_0 + c = 0$ $= 0 + a + \frac{b}{2}$ $\gg 2 \times d_i = 2(a + \frac{b}{2})$ $\therefore d_{start} = 2a + b$ $\boxed{\therefore d_{start} = 2.dy - dx}$

Problem1: Using **Bresenham's Midpoint Line Algorithm**, draw a line from point **(2, 3)** to point **(10, 6)**. Show all steps including the initial decision parameter and how it is updated at each step. Plot the pixels selected by the algorithm.

➔ **Is given,**

$X_0 = 02$ $\quad$ $Y_0 = 3$

$X_1 = 10$ $\quad$ $Y_1 = 6$

Step 1: Calculate Differences	**Step 2:** Initial Decision Parameter
$\gg dx = x1 - x0 = 10 - 2 = 8$ $\gg dy = y1 - y0 = 6 - 3 = 3$	$we\ know,$ $d_{start} = 2.dy - dx$ $= 2 \times 3 - 8$ $= -2 \leq 0 \xrightarrow{E} (3,3)$
Step 3: Find the Next Pixel Values	
3.1 $\therefore d_{new} = d_{old} + dE$ $\therefore d_{new} = -2 + 2dy$ $= -2 + 2 \times 3$ $= 4 > 0 \xrightarrow{NE} (4,4)$	3.2 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 4 + 2(dy - dx)$ $= 4 + 2(3 - 8)$ $= -6 \leq 0 \xrightarrow{E} (5,4)$

3.3 $\therefore d_{new} = d_{old} + dE$ $\therefore d_{new} = -6 + 2dy$ $= -6 + 2 \times 3$ $= 0 \leq 0 \xrightarrow{E} (6,4)$	3.4 $\therefore d_{new} = d_{old} + dE$ $\therefore d_{new} = 0 + 2dy$ $= 0 + 2 \times 3$ $= 6 > 0 \xrightarrow{NE} (7,5)$
3.5 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 6 + 2(dy - dx)$ $= 6 + 2 \times (3 - 8)$ $= -4 \leq 0 \xrightarrow{E} (8,5)$	3.6 $\therefore d_{new} = d_{old} + dE$ $\therefore d_{new} = -4 + 2dy$ $= -4 + 2 \times 3$ $= 2 > 0 \xrightarrow{NE} (9,6)$
3.7 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 2 + 2(dy - dx)$ $= 2 + 2 \times (3 - 8)$ $= -8 \leq 0 \xrightarrow{E} (10,6)$	**Final Pixels Plotted:** (2,3), (3,3), (4,4), (5,4), (6,5), (7,5), (8,5), (9,6), (10,6)

Problem2: Using **Bresenham's Midpoint Line Algorithm**, draw a line from point **(1, 1)** to point **(7, 6)**. Show all steps including the initial decision parameter and how it is updated at each step. Plot the pixels selected by the algorithm.

➔ **Is given,**

$X_0 = 1$ $Y_0 = 1$

$X_1 = 7$ $Y_1 = 6$

Step 1: Calculate Differences	**Step 2:** Initial Decision Parameter
$\gg dx = x1 - x0 = 7 - 1 = 6$ $\gg dy = y1 - y0 = 6 - 1 = 5$ $\gg dy - dx = 5 - 6 = -1$ $\gg 2(dy - dx) = 2(-1) = -2$	***we know,*** $d_{start} = 2.dy - dx$ $= 2 \times 5 - 6$ $= 4 > 0 \xrightarrow{NE} (2,2)$
Step 3: Find the Next Pixel Values	
3.1 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 4 + 2(dy - dx)$ $= 4 - 2$ $= 2 > 0 \xrightarrow{NE} (3,3)$	3.2 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 2 + 2(dy - dx)$ $= 2 - 2$ $= 0 \leq 0 \xrightarrow{E} (4,3)$

3.3	$\therefore d_{new} = d_{old} + dE$ $\therefore d_{new} = 0 + 2dy$ $= 0 + 2 \times 5$ $= 10 > 0 \xrightarrow{NE} (5,4)$	3.4	$\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 10 + 2(dy - dx)$ $= 10 - 2$ $= 8 > 0 \xrightarrow{NE} (6,5)$
3.5	$\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 8 + 2(dy - dx)$ $= 8 - 2$ $= 6 > 0 \xrightarrow{NE} (7,6)$		**Final Pixels Plotted:** (1,1), (2,2), (3,3), (4,3), (5,4), (6,5), (7,6)

<u>Problem3</u>: Using **Bresenham's Midpoint Line Algorithm**, draw a line from point **(2, 1)** to point **(2, 7)**. Show all steps including the initial decision parameter and how it is updated at each step. Plot the pixels selected by the algorithm.

➔ **Is given,**

$X_0 = 2$ $\quad$ $Y_0 = 1$

$X_1 = 2$ $\quad$ $Y_1 = 7$

Step 1: Calculate Differences

$$\gg dx = x1 - x0 = 2 - 2 = 0$$
$$\gg dy = y1 - y0 = 7 - 1 = 6$$

Since $dx = 0$, this is a **vertical line**.

So, Vertical lines aren't handled normally in Bresenham's algorithm because dx = 0, so x doesn't change. The algorithm relies on x-increment to update the decision parameter, which doesn't work for vertical lines. Special handling is required.

Problem4: Using **Bresenham's Midpoint Line Algorithm**, draw a line from point **(9, 5)** to point **(15, 13)**. Show all steps including the initial decision parameter and how it is updated at each step. Plot the pixels selected by the algorithm.

➔ **Is given,**

$X_0 = 09$ $\quad$ $Y_0 = 05$

$X_1 = 15$ $\quad$ $Y_1 = 13$

Step 1: Calculate Differences	**Step 2:** Initial Decision Parameter
$\gg dx = x1 - x0 = 15 - 9 = 6$ $\gg dy = y1 - y0 = 13 - 5 = 8$ $\gg dy - dx = 8 - 6 = 2$ $\gg 2(dy - dx) = 2(2) = 4$	***we know,*** $d_{start} = 2.dy - dx$ $= 2 \times 8 - 6$ $= 10 > 0 \xrightarrow{NE} (10,6)$
Step 3: Find the Next Pixel Values	
3.1 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 10 + 2(dy - dx)$ $= 10 + 4$ $= 14 > 0 \xrightarrow{NE} (11,7)$	3.2 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 14 + 2(dy - dx)$ $= 14 + 4$ $= 18 > 0 \xrightarrow{NE} (12,8)$
3.3 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 18 + 2(dy - dx)$ $= 18 + 4$ $= 22 > 0 \xrightarrow{NE} (13,9)$	3.4 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 22 + 2(dy - dx)$ $= 22 + 4$ $= 26 > 0 \xrightarrow{NE} (14,10)$
3.5 $\therefore d_{new} = d_{old} + dNE$ $\therefore d_{new} = 26 + 2(dy - dx)$ $= 26 + 4$ $= 30 > 0 \xrightarrow{NE} (15,11)$	**Final Pixels Plotted:** (9,5), (10,6), (11,7), (12,8), (13,9), (14,10), (15,11)
This is the output of Bresenham's algorithm for this line. The algorithm ensures that the line is as close as possible to the true line while using only integer arithmetic.	

Derivation of Bresenham's Midpoint Circle Drawing Algorithm

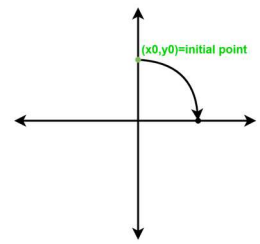
⇨ **We Know,**

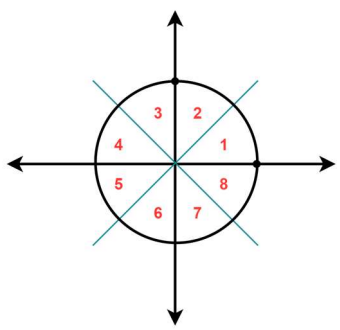
➢ **Circle Equation:**

⇨ The general Circle equation:

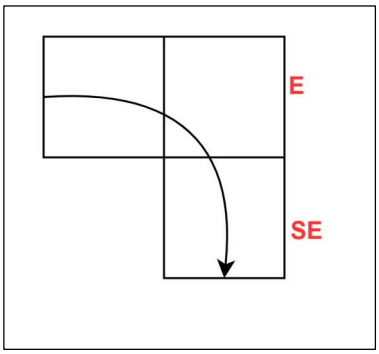
$$\gg x^2 + y^2 = R^2$$

⇨ $\therefore f(x, y) = x^2 + y^2 - R^2$





8-fold Symmetry



Decision Making Using Midpoint

≫**Input:** Start point (x_p, y_p)

≫**We step along x, from x_p to x_1 deciding between two pixels:**

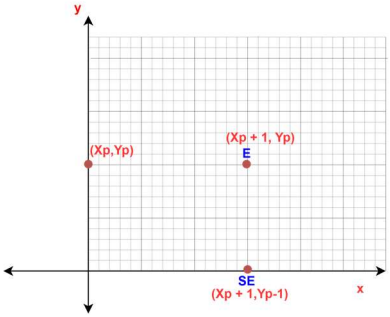
- **E (East)**: $(x_p + 1, y_p)$
- **SE (South_East)**: $(x_p + 1,\ y_p - 1)$

$$\therefore mid = \left(x_p + 1,\ y_p - \frac{1}{2}\right)$$

$\therefore$ f(m) = Decision Making parameter

Or $d = f(m) = f(x_p + 1,\ y_p - \frac{1}{2})$

$$d = (x_p + 1)^2 + (y_p - \tfrac{1}{2})^2 - R^2$$



Calculation for East (d<0)	Calculation for South-East (d>=0)
we know, $mid = \left(x_p + 1,\ y_p - \frac{1}{2}\right)$ $\gg mid_E = \left(x_p + 1 + 1,\ y_p - \frac{1}{2}\right)$ $= \left(x_p + 2\right)^2 + \left(y_p - \frac{1}{2}\right)^2 - R^2$ $\gg \Delta d = f(mid_E) - f(m)$ $= 2x_p + 3$ $\gg h + \Delta d = 2x_p + 3 + h$ $\therefore h_E = 2x_p + 3 + h$ $\boxed{\therefore h_{new} = h_{old} + 2x_p + 3}$	***we know,*** $mid = \left(x_p + 1,\ y_p - \frac{1}{2}\right)$ $\gg mid_{SE} = \left(x_p + 1 + 1,\ y_p - \frac{1}{2} - 1\right)$ $= \left(x_p + 2\right)^2 + \left(y_p - \frac{3}{2}\right)^2 - R^2$ $\gg \Delta d = f(mid_{SE}) - f(m)$ $= 2(x_p - y_p) + 5$ $\gg h + \Delta d = 2(x_p - y_p) + 5 + h$ $\therefore h_{SE} = 2(x_p - y_p) + 5 + h$ $\boxed{\therefore h_{new} = h_{old} + 2(x_p - y_p) + 5}$

Initial Decision Parameters

if,

$$mid_0 = \left(x_0 + 1,\ y_0 - \frac{1}{2}\right)$$

$$\gg d_i = f(mid_0)$$

$$\gg d_i = f\left(x_0 + 1,\ y_0 - \frac{1}{2}\right)$$

$$= f\left((0 + 1), (\mathbf{R} - \frac{1}{2})\right)$$

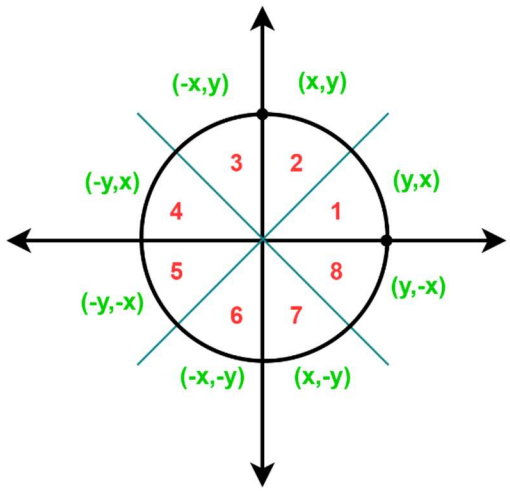
$$= 1^2 + (R - \frac{1}{2})^2 - R^2 = \frac{5}{4} - R$$

$$\gg d_i - \frac{1}{4} = \frac{5}{4} - R - \frac{1}{4}$$

$$\therefore h_{start} = 1 - R$$

$$\boxed{\therefore h_{start} = 1 - R}$$

≫**SO,** 8-fold Symmetry are:



Problem1: Using **Bresenham's Midpoint Circle Algorithm**, For the given equations of circle $(x)^2+(y)^2 = 49$

a. Find the center of the circle
b. Find the radius of the circle
c. Initial coordinate
d. Coordinate for Octant 2
e. Find the coordinates for the other 7 octants.

(a)	**(b)**
≫**is given,** ≫ $(x)^2+(y)^2 = 49$ ≫ $(x)^2+(y)^2 = 7^2$ So, center (x,y) = (0,0)	≫**is given,** ≫ $(x)^2+(y)^2 = 49$ ≫ $(x)^2+(y)^2 = 7^2$ So, radius (r) = 7

(c)

≫**is given,**

≫ $(x)^2+(y)^2 = 49$
≫ $(x)^2+(y)^2 = 7^2$

So, initial coordinate $= (0, r)$
$= (0,7)$

(d)

Step 1: Initial Decision Parameter

we know,

$$h_{start} = 1 - R$$
$$= 1 - 7$$
$$= -6 < 0 \xrightarrow{E} (1,7)$$

Step 2: Find the Next Pixel Values

2.1

$$\therefore h_{new} = h_{old} + 2x_p + 3$$
$$= -6 + 2 \times 0 + 3$$
$$= -3 < 0 \xrightarrow{E} (2,7)$$

2.2

$$\therefore h_{new} = h_{old} + 2x_p + 3$$
$$= -3 + 2 \times 1 + 3$$
$$= 2 \geq 0 \xrightarrow{SE} (3,6)$$

2.3

$$\therefore h_{new} = h_{old} + 2(x_p - y_p) + 5$$
$$= 2 + 2(2 - 7) + 5$$
$$= -3 < 0 \xrightarrow{E} (4,6)$$

2.4

$$\therefore h_{new} = h_{old} + 2x_p + 3$$
$$= -3 + 2 \times 3 + 3$$
$$= 6 \geq 0 \xrightarrow{SE} (5,5)$$

2.5

$$\therefore h_{new} = h_{old} + 2(x_p - y_p) + 5$$
$$= 6 + 2(4 - 6) + 5$$
$$= 7 \geq 0 \xrightarrow{SE} (6,4) \quad [(x > y)]$$

(e)

Octant 2	Octant 6	Octant 1	Octant 5	Octant 3	Octant 7	Octant 4	Octant 8
(0,7)	(-0,-7)	(7,0)	(-7,-0)	(-0,7)	(0,-7)	(-7,0)	(7,-0)
(1,7)	(-1,-7)	(7,1)	(-7,-1)	(-1,7)	(1,-7)	(-7,1)	(7,-1)
(2,7)	(-2,-7)	(7,2)	(-7,-2)	(-2,7)	(2,-7)	(-7,2)	(7,-2)
(3,6)	(-3,-6)	(6,3)	(-6,-3)	(-3,6)	(3,-6)	(-6,3)	(6,-3)
(4,6)	(-4,-6)	(6,4)	(-6,-4)	(-4,6)	(4,-6)	(-6,4)	(6,-4)
(5,5)	(-5,-5)	(5,5)	(-5,-5)	(-5,5)	(5,-5)	(-5,5)	(5,-5)

Problem2: Using **Bresenham's Midpoint Circle Algorithm**, For the given equations of circle $(x+2)^2+(y-3)^2 = 49$

a. Find the center of the circle
b. Find the radius of the circle
c. Initial coordinate
d. Coordinate for Octant 2
e. Find the coordinates for the other 7 octants.

(a)	(b)
≫is given, ≫ $(x+2)^2+(y-3)^2 = 49$ ≫ $(x+2)^2+(y-3)^2 = 7^2$ So, center (x,y) = (-2,3)	≫is given, ≫ $(x+2)^2+(y-3)^2 = 49$ ≫ $(x+2)^2+(y-3)^2 = 7^2$ So, radius (r) = 7

(c)

≫is given,

≫ $(x+2)^2+(y-3)^2 = 49$

≫ $(x+2)^2+(y-3)^2 = 7^2$

$So, initial\ coordinate\ = (0, r) = (0,7)$

So, initial coordinate after shifting $= (0+x, r+y)$

$= (0+(-2), 7+3)$

$= (-2,10)$

(d)

Step 1: Initial Decision Parameter

Let say, initial coordinate = (0,7)

we know,

$$h_{start} = 1 - R$$
$$= 1 - 7$$
$$= -6 < 0 \xrightarrow{E} (1,7)$$

Step 2: Find the Next Pixel Values

2.1

$$\therefore h_{new} = h_{old} + 2x_p + 3$$
$$= -6 + 2 \times 0 + 3$$
$$= -3 < 0 \xrightarrow{E} (2,7)$$

2.2

$$\therefore h_{new} = h_{old} + 2x_p + 3$$
$$= -3 + 2 \times 1 + 3$$
$$= 2 \geq 0 \xrightarrow{SE} (3,6)$$

2.3

$$\therefore h_{new} = h_{old} + 2(x_p - y_p) + 5$$
$$= 2 + 2(2 - 7) + 5$$
$$= -3 < 0 \xrightarrow{E} (4,6)$$

2.4

$$\therefore h_{new} = h_{old} + 2x_p + 3$$
$$= -3 + 2 \times 3 + 3$$
$$= 6 \geq 0 \xrightarrow{SE} (5,5)$$

2.5

$$\therefore h_{new} = h_{old} + 2(x_p - y_p) + 5$$
$$= 6 + 2(4 - 6) + 5$$
$$= 7 \geq 0 \xrightarrow{SE} (6,4) \quad [(x > y)]$$

(e)

Octant 2	Octant 6	Octant 1	Octant 5	Octant 3	Octant 7	Octant 4	Octant 8
(0,7)	(-0,-7)	(7,0)	(-7,-0)	(-0,7)	(0,-7)	(-7,0)	(7,-0)
(1,7)	(-1,-7)	(7,1)	(-7,-1)	(-1,7)	(1,-7)	(-7,1)	(7,-1)
(2,7)	(-2,-7)	(7,2)	(-7,-2)	(-2,7)	(2,-7)	(-7,2)	(7,-2)
(3,6)	(-3,-6)	(6,3)	(-6,-3)	(-3,6)	(3,-6)	(-6,3)	(6,-3)
(4,6)	(-4,-6)	(6,4)	(-6,-4)	(-4,6)	(4,-6)	(-6,4)	(6,-4)
(5,5)	(-5,-5)	(5,5)	(-5,-5)	(-5,5)	(5,-5)	(-5,5)	(5,-5)

After shifting by (-2, 3)

Octant 2	Octant 6	Octant 1	Octant 5	Octant 3	Octant 7	Octant 4	Octant 8
(-2,10)	(-2,-4),	(5,3),	(-9,3),	(-2,10),	(-2,-4),	(-9,3),	(5,3),
(-1,10)	(-3,-4),	(5,4),	(-9,2),	(-3,10),	(-1,-4),	(-9,4),	(5,2),
(0,10)	(-4,-4),	(5,5),	(-9,1),	(-4,10),	(0,-4),	(-9,5),	(5,1),
(1,9)	(-5,-3),	(4,6),	(-8,0),	(-5,9),	(1,-3),	(-8,6),	(5,0),
(2,9)	(-6,-3),	(4,7),	(-8,-1),	(-6,9),	(2,-3),	(-8,7),	(4,-1),
(3,8)	(-7,-2)	(3,8),	(-7,-2)	(-7,8)	(3,-2)	(-7,8)	(3,-2)

MCQ Practice

1. Assigned colors for all three vertices in an OpenGL causes which of the following?

A. Change of overall color of the shape
B. Change in the color at the center of the shape
C. Creating an illusion of colors
D. Creating a shading like effect

2. In coloring, the technique that uses a storage approach to identify the color codes is known as:

A. Direct Coding
B. Lookup Table
C. Rasterization
D. Halftone

3. Which of the following is a property of incremental algorithm?

A. Overlapping pixels
B. Floating point pixel grid
C. Slope
D. Disjoint colors in pixels

4. What's the formula for the decision parameter in East for Bresenham's midpoint circle algorithm:

A. $d_0 + 2x + 3$
B. $d_0 + 2(x - y) + 5$
C. $x^2 + y^2 - R^2$
D. 1 - R

5. In OpenGL, which one is a callback function?

A. glColor3ub()
B. glEnd()
C. glutDisplayFunc()
D. gluFlush()

6. For the equation $(x+5)^2 + (y-7)^2 = 16$ The initial Y co-ordinate for Bresenham's midpoint line algorithm is generally considered:

A. 16
B. 4
C. 5
D. 7

7. The property of Halftoning is:

A. Assigning colors in pixels
B. Dots are inversely proportional
C. Viewing from a distance gives an illusion
D. Both B and C

8. The complementary color of green is:

A. Cyan
B. Black
C. Magenta
D. Yellow

9. When we require more default colors for a system which of the following need not to be applied:

A. Increase in number of bits of color representation
B. Change in default window position
C. Increase of number of pixels per inch
D. Use of additive color model

10. The default value of Left in the co-ordinate system in OpenGL with respect to GluOrtho2D() is:

A. 0
B. -1
C. 1
D. None of the above

11. The number of pixels stored in the frame buffer of a graphics system is known as

A. Depth
B. Resolution
C. Image
D. Pixel

12. Which of the following is correct?

A. Rendering studies algorithms to reflect the light
B. Rendering studies algorithms to transmit the light
C. Computer graphics studies the manipulation of visual and geometric information using non-computational techniques.
D. Vector graphics are composed of paths

13. Which one is true?

A. Y = G + R
B. C = R + G
C. Y = W – G
D. M = R + G

14. Which one is not a format of Raster graphics?

A. TIFF
B. PCX
C. GIF
D. PICT

15. Converting Raster Image to Vector image is called

A. Scan Conversation
B. Rasterization
C. Halftoning
D. Dithering

16. To use Polygon primitive, the least number of points needed?

A. 3
B. 1
C. 2
D. None of the above

17. What can OpenGL render?

A. Primitives
B. Bitmaps
C. Texture
D. All of the above

18. For a particular situation you are using the formula x=x+1 and y=round(y+m) for DDA. What could be the situation?

A. m>1
B. m=1
C. m=0
D. m<1

19. The binary for color intensity values (136, 85, 212) is:

A. (10001000, 01010101, 11010100)
B. (10001000, 01010101, 10101110)
C. (00001000, 01010101, 11010100)
D. (10000010, 01010101, 11010100)

20. $(x-3)^2 + (y-5)^2 = 49$, considering the equation, which one of the following would be the initial Y coordinate according to Bresenham's midpoint circle algorithm?

A. 1
B. 3
C. 7
D. 8

21. In CMYK, the key color is

A. Magenta
B. Cyan
C. Black
D. Yellow

22. Each color has its ____ different intensity level

A. 8
B. 256
C. 100
D. 200

23. Which one is not a format of Vector graphics?

A. AI
B. PCX
C. CGM
D. PICT

24. The DDA algorithm is _____ dependent.

A. Resolution
B. Orientation
C. Projection
D. Transformation

25. Scan conversion takes place after

A. Rendering
B. Mapping
C. Mapping Shading
D. Clipping

26. An image is visible when:

A. A bitmap is created and shaded
B. Bitmap is converted to vector image
C. Bitmap is formed
D. Bitmap is rasterized

27. Which of the following needs RGB values?

A. Position of a pixel
B. Background Color
C. Window Position
D. Window Title

28. Which function is meant for looping continuously through glut events?

A. glutDisplayFunc()
B. glutInit(&argc, argv)
C. glutMainLoop()
D. display()

29. Total number of colors in a colour palette depends on:

A. Pixel numbers in an image
B. The color space used
C. The color model used
D. Total number of bits for each color

30. Which of the following is an argument for the function glClearColor():

A. Opacity
B. Intensity colors
C. Vertex value
D. GL Primitive type

31. How many vertices are required to create a four-sided box with lines in OpenGL?

A. 4
B. 2
C. 8
D. Not possible

32. How do we create a primitive circle using OpenGL?

A. Polygon
B. Points
C. Lines
D. All of the above

33. Which one is NOT a format of raster graphics?

A. GIF
B. PICT
C. TIFF
D. GIF

34. Circle is also known as a _____ symmetry:

A. 4 way
B. 6 way
C. 8 way
D. 10 way

35. Which of the following is one of the steps for the midpoint circle algorithm?

A. $d = 3 - 2r$
B. $d = 1 - R$
C. $m = dy/dx$
D. $ax + by + c = 0$

36. Which of the following is a valid formula for line drawing?

A. $x = round(x + m)$
B. $y = y + 2$
C. $2dy$
D. $ax + by + c = 0$

37. For generating a line, which features do we need to consider?

A. End point
B. Floating point arithmetic
C. Avoiding floating point arithmetic
D. All of the above

38. Which one is NOT an example of Computer Graphics in real life?

A. In fields of entertainment, such as video games and animation
B. In fields of computer-aided design
C. Both A and B
D. Only A

39. The quality of images depends on:

A. Number of pixels
B. Colors assigned to each pixel
C. Display device quality
D. All of the above

40. Halftone breaks up an image into a series of

A. Scan code
B. Shade
C. Shape
D. None

41. Which one is not a format of Raster graphics?

A. TIFF
B. PCX
C. PICT
D. GIF

42. In coloring, the technique that uses a storage approach to identify the color codes is:

A. Direct Coding
B. Lookup Table
C. Rasterization
D. Halftone

43. What is the formula for the decision parameter in East for Bresenham's midpoint circle algorithm?

A. $d_0 + 2x + 3$
B. d0 + 2(x - y) + 5
C. 1 - R
D. None

44. If point A (2, 3) belongs to a line having m = 2.5, what will be the next point if DDA is applied?

A. B(3, 3)
B. B(2, 4)
C. B(3, 4)
D. B(4, 4)

45. If d >= 0, then next which point is selected according to midpoint circle algorithm?

A. East
B. North-East
C. South-East
D. Both A & B

46. Which one is a callback function?

A. glColor3ub()
B. glEnd()
C. glutDisplayFunc()
D. glFlush()

47. For the equation $(x+5)^2 + (y-7)^2 = 16$ the initial Y-coordinate for Bresenham's midpoint line algorithm is generally considered:

A. 16
B. 4
C. -5
D. 7

48. More than 4 vertices in a primitive means:

A. Line
B. Quad
C. Polygon
D. None

49. An image composed of individual pixels arranged in a grid is referred to as:

A. Vector graphic
B. Raster image
C. Polygon mesh
D. Bezier curve

50. For a vector image the mathematical formula is used:

A. To find the pixels
B. To find the color
C. To find the coordinates that are mapped to pixels
D. None

51. For a raster image pixel can be arranged in a regular

A. ID Grid
B. 2D Vector
C. Bitmap
D. Coordinates

52. $(x-1)^2+(y-1)^2 = 25$, considering the equation, which one of the following would be the initial point according to Bresenham's midpoint circle algorithm?

A. (0, 25)
B. (0, 5)
C. (1, 1)
D. (-1, -1)

53. If $d > 0$, then which one is true for midpoint line algorithm?

A. tan = 2dy - dx
B. East point is selected if $d > 0$
C. $(\Delta d)E = 2dy$
D. $(\Delta d)NE = 2(dy - dx)$

54. Which of the following is a coloring technique?
A. Direct Coding
B. Lookup Table
C. Both A and B
D. None

Part B: True/False

1. Pixels in an image can be reproduced at a fixed size without the appearance of visible dots or squares.

2. CMYK ranges from 0 to 100%.

3. Individually each pixel carries meaning.

4. Incremental algorithm has 3 requirements.

5. (R, G, B) ranges from 0 to 255.

6. Vector images have no fixed intrinsic resolution.

7. In computer graphics, a color palette is an infinite set of colors.

8. Colors perceived in subtractive models are the result of translated light.

9. GL_CIRCLE is not a geometric primitive.

10. In midpoint line algorithm, incremental calculations are made based on preceding step to find next y value, in order to form an exact approximation to a straight line between two points.

11. Magenta is complementary color of green.

12. Resolution in raster graphics is measured in dpi, or dots per inch. The lower the dpi, the better the resolution.

13. To use polygon primitive at most 3 points are needed.

14. Raster graphics are bitmaps.

15. CMYK stores color values for cyan, magenta, yellow, black.

16. Raster image is better for designing a logo

17. To create polygon primitive at most 3 points are needed

18. Raster images use mathematical calculations

19. Additive color model uses CMY color space

20. Raster images use mathematical calculations

21. Lookup table uses a table like structure to find color codes

22. Additive color models are used in printing

23. Vectors images are significantly smaller than Raster Graphics

24. Resolution on an image is measured in Dots per inch

25. Additive color model uses CMY color space

26. gluOrtho2D() is used to fix the window size

27. Vector graphics are bitmap

28. Subtractive color model utilizes transmitted light for showing an object

29. The number of dots per inch of ink on a printed page determines the resolution of the printer

30. Lookup table looks into a table like structure to find the color codes